Grading Criteria

School Events & Notification Center

Functionality (40 points)

#	Criterion	Points
1	**User authentication** and **role-based access control** (student vs organizer): registration/login as documented; **403** (or equivalent) when a role tries forbidden actions; students cannot see other users' registrations.	10
2	**Events:** organizers can create/update **draft** events; **publish** (DRAFT → PUBLISHED); **cancel or close** an event with **documented** behavior for existing registrations; students only see/list **published** events; event **ownership** (organizer_id) enforced.	10
3	**Registration core:** register for an event → **confirmed** if capacity available, else **waitlist** with **FIFO** ordering; **no overbooking** of confirmed seats under concurrency (locking, transaction, or equivalent); student can **cancel** own registration per team's documented rule; **automatic waitlist promotion** when a confirmed seat is freed.	15
4	**Organizer visibility:** list **confirmed** registrations and **ordered waitlist** for **own** events only; reasonable event detail for capacity/waitlist counts.	5

Backend (30 points)

#	Criterion	Points
1	**Event-driven processing:** domain changes enqueue work; **separate worker process** (not the HTTP server thread) consumes a **queue** (DB notification_jobs / outbox, Redis, RabbitMQ, etc.); producer/consumer clearly separated in repo and README.	10
2	**Database:** relational schema for users, events, registrations, notification jobs (as applicable); **migrations**; constraints/uniqueness (e.g. one active registration per user per event);	10
3	**RESTful API:** resources and HTTP methods match the requirements of the project (auth, events, publish/cancel, registrations, registrations/me, organizer lists); consistent **status codes** and error shape; authentication on protected routes.	10

Additional features (25 points)

#	Criterion	Points
1	**Domain events:** at least **three distinct** queued job/event type values (e.g. RegistrationConfirmed, RegistrationWaitlisted, WaitlistPromoted) end-to-end through the worker.	10
2	**Notifications:** real **email** (third-party or SMTP) and persistent **notification log** (DB table and/or structured console) proving delivery attempts.	5
3	**Reliability:** job states (pending / processing / sent / failed or equivalent); **retries** or clear max-attempt policy; failures visible (logs).	5

4	**Idempotency:** clear strategy so **retries do not duplicate** user-visible notifications (unique key, dedupe table, or equivalent).	5

Frontend (15 points)

#	Criterion	Points
1	**Student experience:** browse published events; register / cancel; view **own** registrations and waitlist position (or status).	5
2	**Organizer experience:** manage own events (create/edit/publish); view registrations and waitlist for own events.	5
3	**UX polish: preview** event as students would see it before publish **or** clear role-based navigation / empty states / basic responsiveness.	5

REST endpoints (10 points)

#	Criterion	Points
1	**Complete and functional** implementation of the **core** REST surface from the project documnetation (auth, events including publish/cancel, registrations including cancel and “me”, organizer registration/waitlist reads); behavior must match the spec’s intent.	10

Security (15 points)

#	Criterion	Points
1	**Passwords:** secure storage using **slow hashing** - not reversible “encryption” of passwords.	5
2	**Input validation** and safe query/data access patterns to reduce **injection** (like SQL injection) and obvious **XSS** (if any HTML is rendered).	5
3	**Authorization:** every sensitive read/write checks **role and ownership** (event organizer, registration owner); mechanisms to control access to sensitive data.	5

Scalability & design (15 points)

#	Criterion	Points
1	Design considerations for scalability to handle a large number of users and items	5
2	Optimization of the database for performance and scalability.	5
3	Scalability of the project architecture to handle increased traffic and demand.	5

Code quality (10 points)

#	Criterion	Points
1	Clean, organized project layout (API vs worker, config, migrations).	3
2	Consistent naming and formatting (formatter/linter optional but welcome).	3
3	Comments or short docs where logic is non-obvious (e.g. waitlist transaction).	2
4	Modular structure and reuse (shared types/DTOs, shared DB access patterns).	2

Presentation (10 points)

#	Criterion	Points
1	Clarity and coherence of the project presentation (problem, solution, architecture).	5
2	Demonstration of **key** functionalities (flows: publish → register / waitlist → cancel → promotion → **async notification** visible) and features with clear explanations.	5

Total: 170 points